PROGRAMACIÓN AVANZADA · BACHILLERATO TECNOLÓGICO · UTU

Actividades: Modificar el MiniCRUD

Tres ejercicios para extender el proyecto base agregando, cambiando y reorganizando atributos a través de todas las capas MVC.

Prof. Elizabeth Izquierdo

CONTENIDO

ACTIVIDAD 1 **Agregar el atributo sueldo (Referencia resuelta)** 2

ACTIVIDAD 2 **Agregar el atributo email (Individual)** 4

ACTIVIDAD 3 **Reemplazar sueldo por categoria con <select> (Desafío)** 6

// ACTIVIDAD 1 — Agregar el atributo sueldo (Referencia resuelta)

Agregar el atributo sueldo

De minicrud0 → minicrud0-mod1 · Referencia resuelta — estudiar antes de las siguientes

REFERENCIA RESUELTA | DOUBLE | 4 ARCHIVOS

PROYECTO BASE Proyecto base (minicrud0)

OBJETIVO

El proyecto base (`minicrud0`) solo guarda **código** y **nombre** de cada persona. La tarea es agregar el atributo **sueldo** (número decimal) y hacer que aparezca en el formulario de alta y en la tabla de listado.

PersonaVO: ~~cod, nombre~~ → cod, nombre, sueldo

// PREGUNTAS ORIENTADORAS — RESPONDER ANTES DE TOCAR CÓDIGO

1. ¿En qué clase Java se define la estructura de datos de una Persona? ¿Cómo se llama el patrón de diseño que usan las clases que terminan en `VO` ?
2. ¿Qué tipo de dato Java se usaría para representar un sueldo con decimales? ¿Por qué no se usaría `int` ?
3. ¿Para qué sirve el *getter* de un atributo? ¿Qué relación tiene con la expresión `${p.sueldo}` que se va a escribir en el JSP?
4. ¿Qué archivo Java recibe los datos que el usuario envía desde el formulario? ¿Por qué es necesario usar `Double.parseDouble()` para el sueldo pero no para el nombre?
5. ¿Cuántos archivos JSP hay que modificar? ¿Cuál es la diferencia entre el formulario y la lista?
6. ¿Por qué **no** es necesario modificar `PersonaDAO.java` ?

// ARCHIVOS INVOLUCRADOS

Archivo	Capa	¿Qué cambia?	Estado
PersonaVO.java	Modelo · VO	Agregar atributo, constructor, getter/setter, toString	Modificar
PersonaControladorServlet.java	Controlador	Leer parámetro `sueldo` del request y pasarlo al constructor	Modificar
persona-form.jsp	Vista · Formulario	Agregar campo `<input>` para sueldo	Modificar
persona-lista.jsp	Vista · Listado	Agregar columna Sueldo en header y en filas	Modificar
PersonaDAO.java	Modelo · DAO	Sin cambios — la lógica de la lista no depende de los atributos	Sin cambios

// PASOS

01

PERSONAVO.JAVA

Agregar el atributo y actualizar constructor, getter, setter y toString

Declarar `private double sueldo`. El constructor con parámetros también debe recibir el nuevo dato.

PersonaVO.java — cambios

```java
// Atributo nuevo
private double sueldo;

// Constructor actualizado (3 parámetros)
public PersonaVO(int cod, String nombre, double sueldo) {
    this.cod    = cod;
    this.nombre = nombre;
    this.sueldo = sueldo;   // ← nueva línea
}

// Getter y setter
public double getSueldo() { return sueldo; }
public void setSueldo(double sueldo) { this.sueldo = sueldo; }
```

Pregunta de verificación: Si el JSP llama a `${p.sueldo}`, ¿a qué método Java llama internamente la Expression Language?

02

PERSONACONTROLADORSERVLET.JAVA

Leer el parámetro sueldo del request

En el método privado `agregar()`, el Servlet lee los datos del formulario con `request.getParameter()`. Todos los parámetros llegan como **String**; hay que convertir el sueldo a `double`.

PersonaControladorServlet.java — método agregar()

```java
private void agregar(HttpServletRequest request, ...) throws IOException {
    int    codigo = Integer.parseInt(request.getParameter("codigo"));
    String nombre = request.getParameter("nombre");
    double sueldo = Double.parseDouble(request.getParameter("sueldo")); // ← nuevo
    dao.agregar(new PersonaVO(codigo, nombre, sueldo));
    response.sendRedirect(...);
}
```

Verificación: ¿Qué excepción lanzaría esta línea si el usuario escribe letras en el campo sueldo?

03

PERSONA-FORM.JSP

Agregar el campo de sueldo en el formulario

El atributo `name` del input debe coincidir **exactamente** con el nombre que el Servlet lee con `request.getParameter()`. Se usa `type="number"` y `step="0.01"` para permitir decimales.

persona-form.jsp — dentro del <form>

```
<label>Nombre</label>
<input type="text" name="nombre" required />

<label>Sueldo</label>
<input type="number" name="sueldo" step="0.01" required />

<button class="btn" type="submit">Guardar</button>
```

Verificación: Si se cambia el `name` a `"salario"` pero en el Servlet se sigue usando `getParameter("sueldo")`, ¿qué valor llega? ¿El error sería en compilación o en ejecución?

04

PERSONA-LISTA.JSP

Mostrar la columna Sueldo en la tabla

Hay dos lugares: el **header** de la tabla (`<th>`) y dentro del `<c:forEach>`, una celda `<td>` por cada persona.

persona-lista.jsp — dos cambios

```
<!-- 1) En el <thead> -->
<th>Código</th>
<th>Nombre</th>
<th>Sueldo</th>
<th>Acciones</th>

<!-- 2) Dentro del <c:forEach var="p"> -->
<td>${p.codigo}</td>
<td>${p.nombre}</td>
<td>${p.sueldo}</td>
```

Verificación: ¿Qué pasaría si se agrega el `<th>` pero se olvida el `<td>` dentro del forEach?

PARA VERIFICAR QUE TODO FUNCIONA

Desplegar el proyecto, entrar a *Agregar Persona*, completar el formulario con los tres campos y guardar. La tabla de listado debe mostrar código, nombre y sueldo. Si algo no aparece, revisar que los `name` del form coincidan con los `getParameter()` del Servlet.

// ACTIVIDAD 2 — Agregar el atributo email (Individual)

Agregar el atributo email

De minicrud0-mod1 → minicrud0-mod2 · Actividad individual — sin solución adjunta

INDIVIDUAL STRING 4 ARCHIVOS

PROYECTO BASE Proyecto base (minicrud0-mod1)

OBJETIVO

Partiendo del proyecto con `sueldo` ya funcionando, agregar el atributo **email** (texto libre) de cada persona. Debe aparecer en el formulario de alta y en la tabla de listado.

PersonaVO: ~~cod, nombre, sueldo~~ → cod, nombre, sueldo, email

// PREGUNTAS ORIENTADORAS — RESPONDER ANTES DE TOCAR CÓDIGO

1. ¿Qué tipo de dato Java se usaría para almacenar un email? ¿Es necesario convertirlo con algún método como `Integer.parseInt()` o `Double.parseDouble()`? ¿Por qué?
2. Comparando con la actividad anterior: ¿en qué se diferencia agregar un atributo `String` de agregar un `double`? ¿Qué pasos son exactamente iguales y cuál cambia?
3. El HTML tiene un `type="email"` para inputs de correo electrónico. ¿Qué ventaja tiene usarlo en lugar de `type="text"`? ¿Cambia algo en el servidor?
4. Listar los 4 archivos a modificar y describir en una línea qué va a cambiar en cada uno, *antes de abrir el editor*.
5. Si se agrega el campo en el formulario pero se olvida actualizar el constructor de `PersonaVO`, ¿el proyecto compila? ¿Qué error se vería?

// ARCHIVOS INVOLUCRADOS

Archivo	Capa	¿Qué hay que hacer?
PersonaVO.java	Modelo · VO	Declarar `private String email`, actualizar constructor, agregar getter y setter, actualizar `toString()`
PersonaControladorServlet.java	Controlador	En el método `agregar()`: leer `request.getParameter("email")` y pasarlo al constructor
persona-form.jsp	Vista · Formulario	Agregar `<label>` e `<input type="email">` con `name="email"`
persona-lista.jsp	Vista · Listado	Agregar `<th>Email</th>` en el header y `<td>${p.email}</td>` en cada fila

// PASOS

01 PERSONAVO.JAVA

Agregar el atributo email y actualizar el VO

Seguir el mismo patrón de la actividad anterior. Declarar el atributo, completar el constructor, agregar el getter y el setter, y actualizar el `toString()`.

Pista — estructura esperada

```
private String email;    // ← declarar

// Constructor — ¿cuántos parámetros tiene ahora?
public PersonaVO(int cod, String nombre, double sueldo, String email) {
    // completar...
}

// Getter y setter — igual que sueldo pero con String
```

Reflexión: ¿Por qué el getter se llama `getEmail()` y no `devolverEmail()`? ¿Qué convención de nombres establece Java para los getters?

02 PERSONACONTROLADORSERVLET.JAVA

Leer el email del request en el método agregar()

Localizar el método privado `agregar()`. Leer el nuevo parámetro y pasarlo al constructor de `PersonaVO`.

DIFERENCIA CLAVE CON EL SUELDO

Para el sueldo se usó `Double.parseDouble()`. Para el email, `request.getParameter("email")` ya devuelve un `String`, así que se puede usar directamente. Sin conversión.

Reflexión: ¿Qué pasaría si el usuario deja el campo email vacío y el campo no tiene `required` en el JSP? ¿Qué valor recibiría el Servlet?

03 PERSONA-FORM.JSP

Agregar el campo email en el formulario

Copiar el bloque label+input del nombre y adaptarlo para email. Usar `type="email"`: el navegador validará el formato automáticamente antes de enviar el formulario.

Reflexión: ¿El atributo `type="email"` del input HTML valida el email también en el servidor? ¿Qué pasaría si alguien envía el formulario con una herramienta como Postman sin esa validación?

04 PERSONA-LISTA.JSP

Mostrar la columna Email en la tabla

Dos lugares: agregar el `<th>` en el encabezado de la tabla y agregar el `<td>${p.email}</td>` dentro del `<c:forEach>`.

Reflexión: Si la tabla tiene 3 columnas en el `<thead>` pero 4 en cada fila del `<tbody>`, ¿qué se vería visualmente?

> VERIFICACIÓN FINAL
>
> Agregar una persona con email. Verificar que aparezca en la lista. Luego intentar agregar una persona con el campo email vacío o sin el formato correcto y observar qué hace el navegador.

// PREGUNTAS DE CIERRE — PARA DISCUTIR EN CLASE

1 ¿Podría existir una Persona sin email? ¿Cómo se modelaría diferente si el email fuera opcional?

2. Ahora que `PersonaV0` tiene 4 atributos, ¿qué otros constructores podrían ser útiles?
3. Si el proyecto tuviera base de datos real en vez de un ArrayList en memoria, ¿habría que tocar la clase `PersonaDAO` al agregar el email? ¿Por qué?

// ACTIVIDAD 3 — Reemplazar sueldo por categoria con <select> (Desafío)

Reemplazar sueldo por categoria con <select>

De minicrud0-mod2 → minicrud0-mod3 · Desafío — combina agregar y quitar

DESAFÍO | QUITAR ATRIBUTO | AGREGAR ATRIBUTO | SELECT HTML

PROYECTO BASE Proyecto base (minicrud0-mod2)

OBJETIVO

Esta actividad es diferente: no solo se agrega un atributo nuevo, sino que se **reemplaza** uno existente. El atributo `sueldo` (double, valor libre) se elimina y en su lugar aparece `categoria` (String con tres opciones fijas: *"Junior"*, *"Semi-Senior"* y *"Senior"*). En el formulario, en lugar de un campo de texto libre, el usuario elige de una lista desplegable.

PersonaVO: ~~sueldo (double)~~ → categoria (String)

Formulario: ~~<input number>~~ → <select>

// PREGUNTAS ORIENTADORAS — RESPONDER ANTES DE TOCAR CÓDIGO

1. ¿Qué diferencia hay entre *agregar* un atributo y *reemplazarlo*? ¿Cuántos pasos adicionales implica quitar algo?
2. ¿Qué tipo de dato Java se usaría para `categoria` si las opciones son "Junior", "Semi-Senior" y "Senior"? ¿Podría ser un `int`? ¿Cuáles serían las ventajas y desventajas de usar un `enum`?
3. El elemento HTML `<select>` tiene varias etiquetas `<option>` adentro. ¿Cómo llega el valor seleccionado al Servlet? ¿Con qué método se lee?
4. Cuando el usuario elige una opción de un `<select>`, ¿el valor que llega es el texto del `<option>` o el atributo `value`? ¿Qué pasa si no se pone el atributo `value`?
5. ¿Es necesario modificar `PersonaDAO.java` para este cambio? Justificar la respuesta.
6. ¿Qué ventaja tiene ofrecer opciones fijas (un select) frente a dejar que el usuario escriba la categoría libremente?

// ARCHIVOS INVOLUCRADOS

Archivo	¿Qué hay que hacer?
PersonaVO.java	Quitar `double sueldo` → agregar `String categoria`. Actualizar constructor, getters/setters y toString.
PersonaControladorServlet.java	Quitar lectura de `sueldo`. Leer `categoria` con `getParameter()` sin conversión.
persona-form.jsp	Reemplazar el `<input type="number">` por un `<select>` con 3 opciones.
persona-lista.jsp	Cambiar la columna "Sueldo" por "Categoría" y la celda `${p.sueldo}` por `${p.categoria}`.

// PASOS

01

PERSONAVO.JAVA

Quitar sueldo, agregar categoria

Eliminar la declaración de `double sueldo` y todos sus métodos. Agregar `String categoria` con su getter y setter. Actualizar el constructor para que reciba **cod, nombre, email, categoria** (en ese orden).

PersonaVO.java — antes y después

```
// ── QUITAR ──────────────────────────
private double sueldo;
public PersonaVO(int cod, String nombre, double sueldo, String email) {...}
public double getSueldo() { return sueldo; }
public void setSueldo(double sueldo) { this.sueldo = sueldo; }

// ── AGREGAR ─────────────────────────
private String categoria;
public PersonaVO(int cod, String nombre, String email, String categoria) {...}
public String getCategoria() { return categoria; }
public void setCategoria(String categoria) { this.categoria = categoria; }
```

Reflexión: ¿Qué pasaría si se olvida quitar el getter de `sueldo` pero sí se quita el atributo? ¿El compilador avisa?

02

PERSONACONTROLADORSERVLET.JAVA

Actualizar el método agregar() del Servlet

Quitar la línea que leía el sueldo y convertía con `Double.parseDouble()`. En su lugar, leer `categoria` directamente como String — igual que el nombre o el email.

Método agregar() — resultado esperado

```
private void agregar(...) throws IOException {
    int    codigo    = Integer.parseInt(request.getParameter("codigo"));
    String nombre    = request.getParameter("nombre");
    String email     = request.getParameter("email");
    String categoria = request.getParameter("categoria");   // ← nuevo
    // double sueldo = Double.parseDouble(...); // ← eliminar
    dao.agregar(new PersonaVO(codigo, nombre, email, categoria));
    response.sendRedirect(...);
}
```

Reflexión: ¿Por qué el valor que llega desde un `<select>` se lee igual que el de un `<input>`? ¿Cómo sabe el Servlet de dónde viene?

03

PERSONA-FORM.JSP

Reemplazar el input numérico por un <select>

Borrar el bloque de `label + input[sueldo]` y escribir un `<select>` con tres `<option>`. El atributo `name` del `<select>` debe ser *"categoria"*.

persona-form.jsp — bloque a reemplazar

```
<!-- QUITAR esto -->
<label>Sueldo</label>
<input type="number" name="sueldo" step="0.01" required />

<!-- AGREGAR esto -->
<label>Categoría</label>
<select name="categoria" required>
  <option value="Junior">Junior</option>
  <option value="Semi-Senior">Semi-Senior</option>
  <option value="Senior">Senior</option>
</select>
```

> DESAFÍO EXTRA
>
> Investigar el atributo `selected` en un `<option>`. ¿Cómo se haría para que "Semi-Senior" aparezca seleccionado por defecto?

04

PERSONA-LISTA.JSP

Cambiar columna "Sueldo" por "Categoría"

Dos reemplazos: el texto del `<th>` en el header y la expresión EL en el `<td>` de cada fila.

persona-lista.jsp — cambios

```
<!-- Header -->
<th>Sueldo</th>
<th>Categoría</th>

<!-- Celda en el forEach -->
<td>${p.sueldo}</td>
<td>${p.categoria}</td>
```

Reflexión: ¿Qué pasaría si se escribe `${p.Categoria}` con mayúscula? ¿La Expression Language es sensible a mayúsculas? ¿A qué método llama internamente?

VERIFICACIÓN COMPLETA

1. Entrar a *Agregar Persona*. El campo Sueldo ya no debe estar; debe aparecer un desplegable con las 3 categorías.
2. Elegir una categoría y guardar. En la lista debe aparecer la categoría correcta en la columna.
3. Agregar personas con diferentes categorías y verificar que cada una guarda la suya.

// **PREGUNTAS DE CIERRE** — PARA DISCUTIR O INVESTIGAR

1. ¿Qué ventajas tendría usar un `enum` de Java (por ejemplo `Categoria.JUNIOR`) en lugar del String `"Junior"`? ¿Cuáles serían las desventajas?
2. Si en el futuro la lista de categorías cambia (por ejemplo, se agrega "Trainee"), ¿en qué archivos habría que tocar? ¿Cómo se podría centralizar esa lista en un solo lugar?
3. Ahora que el proyecto tiene `cod`, `nombre`, `email` y `categoria`: si se quisiera que el email sea único (que no se repita), ¿en qué capa del MVC se implementaría esa validación — en el VO, en el DAO o en el Servlet? ¿Por qué?